

Evolutionary Algorithms & Other Nature Inspired Algorithms

SE3062 – Intelligent Systems

Sri Lanka Institute of Information Technology
(SLIIT)

Author - Jeewaka Perera

Optimization Methods Landscape

- Exact methods: LP, DP – optimal but structured
- Heuristic search: A*, Greedy – fast but not always optimal
- Metaheuristics: EA, Simulated Annealing – flexible, adaptable
 - EA is a metaheuristic for global search

Motivation: Why Evolutionary Algorithms?

- Many real-world problems are too complex for exact methods:
 - Large search spaces (NP-hard problems).
 - Nonlinear, discontinuous, noisy objective functions.
- Traditional methods:
 - LP, DP → great for structured, small/medium problems.
 - But fail when constraints or dimensions explode.
- Metaheuristics (EA, PSO, ACO):
 - Explore large spaces.
 - Provide good-enough solutions (not exact).
- Example:
 - Scheduling
 - TSP
 - feature selection in ML.

Biological Inspiration

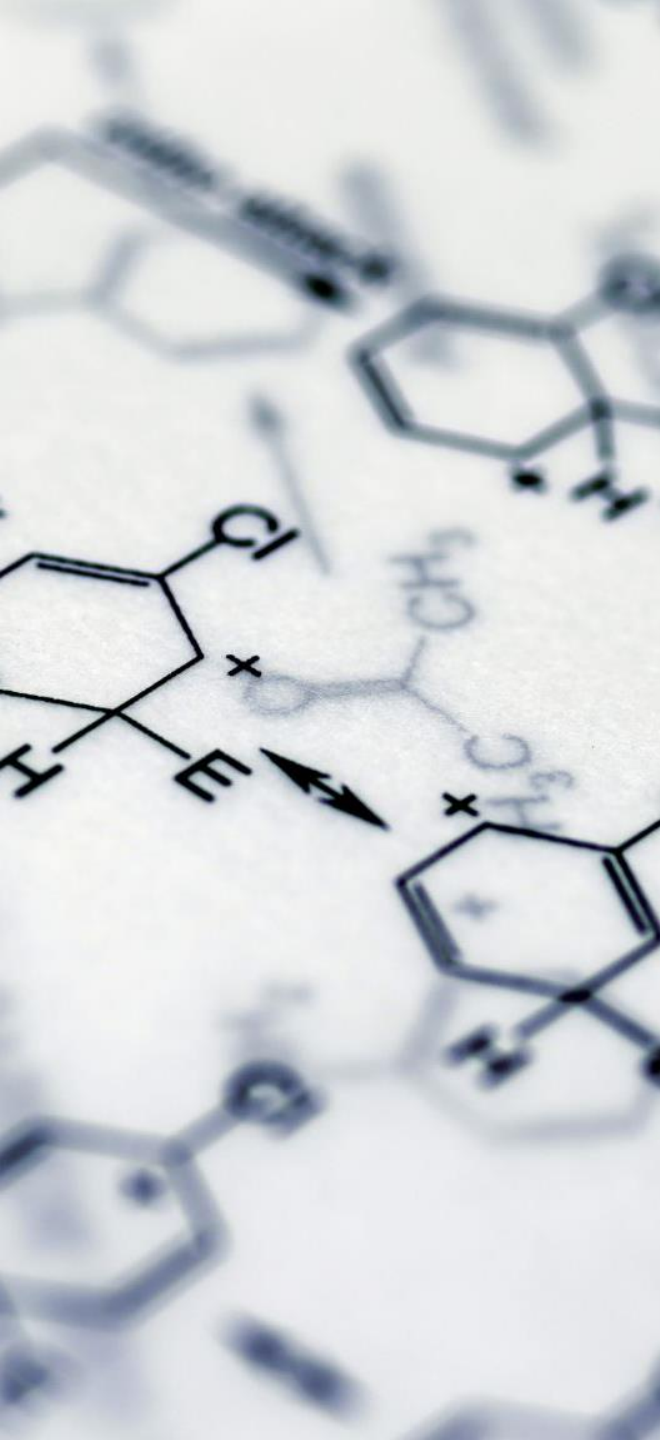
Darwin's Principle: survival of the fittest → optimization.

Mapping to optimization:

- Individual ↔ candidate solution
- Population ↔ set of solutions
- Fitness ↔ objective quality
- Gene ↔ decision variable
- Chromosome ↔ encoded solution

Natural processes used:

- Selection → best survive.
- Crossover → combine good solutions.
- Mutation → introduce diversity.

The background of the slide features a blurred image of chemical structures, including a cyclohexane ring with a chlorine atom and a double bond, and a benzene ring. A double-headed arrow points between two structures, suggesting a reaction or transformation.

Types of EA

- **Evolutionary Programming (EP):**
 - Real-valued vectors, only mutation used.
- **Evolution Strategies (ES):**
 - Real-coded, adaptive mutation rates.
- **Genetic Algorithms (GA):**
 - Binary/permutation encoding, crossover + mutation.
- **Genetic Programming (GP):**
 - Programs (trees) as individuals.

Evolutionary Algorithms (EA)

Population-based metaheuristics inspired by evolution

Steps:

- Initialize
- Evaluate
- Select
- Crossover, mutate
- Repeat

Goal: approximate global optimum

General EA Framework

```
Initialize population P
while not termination_condition:
    Evaluate fitness of each individual in P
    Select parents based on fitness
    Apply crossover to produce offspring
    Apply mutation to introduce diversity
    Form new generation P
Return the best solution found
```

- Shared by all EA variants.
- GA is the most popular implementation.

GENETIC ALGORITHM



Genetic Algorithms (GA)

Invented by John Holland (1975).

Operates on a population of solutions.

Uses chromosomes to encode candidate solutions.

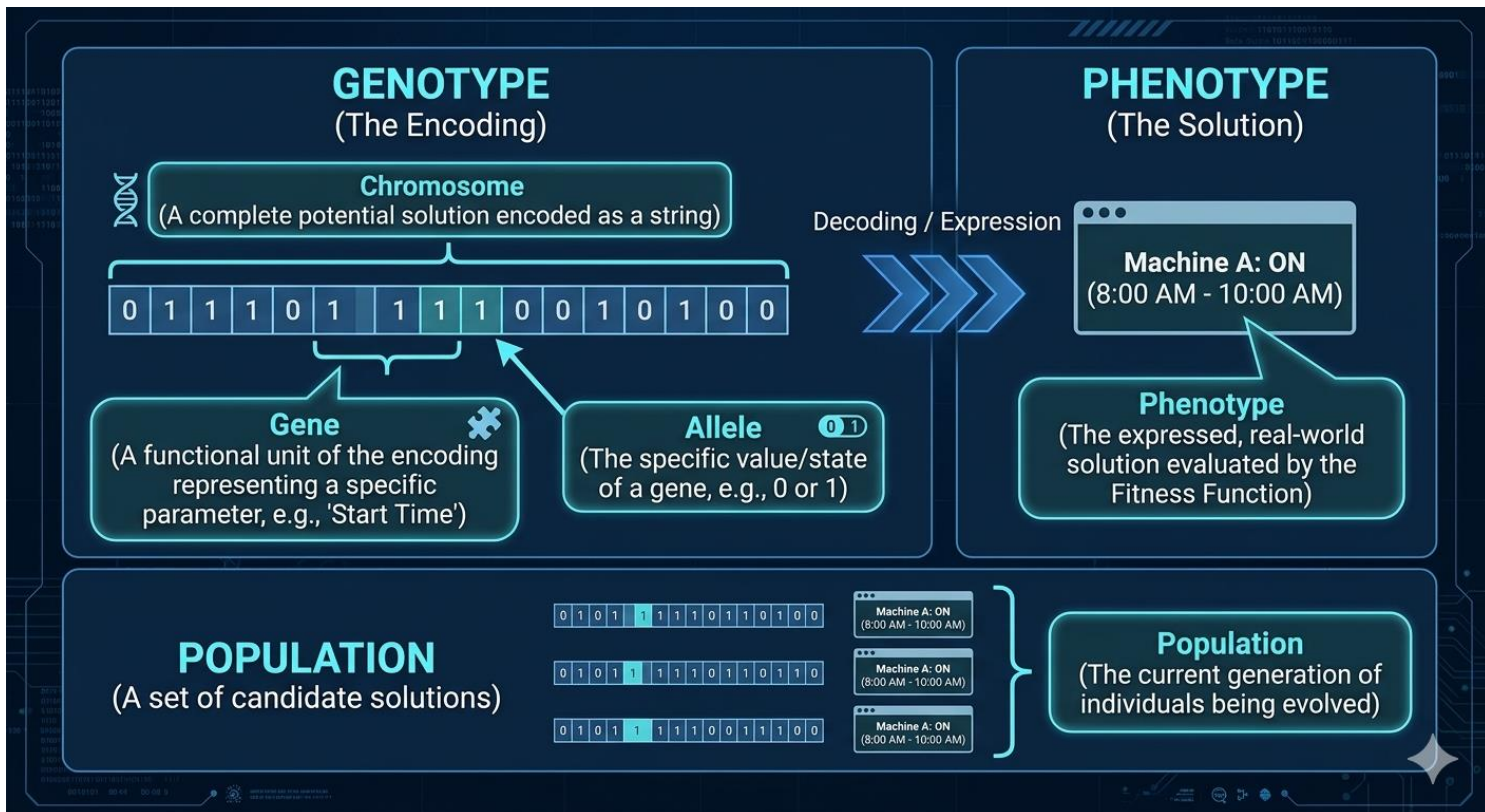
Applies crossover and mutation to create diversity.

Guided by fitness function.

Termination: after fixed generations or when solution is "good enough."

Chromosomes & Representation

- Chromosome: encoded solution.
- Gene: a unit (bit, number, symbol).
- Allele: value of a gene.
- Phenotype: decoded real-world meaning.

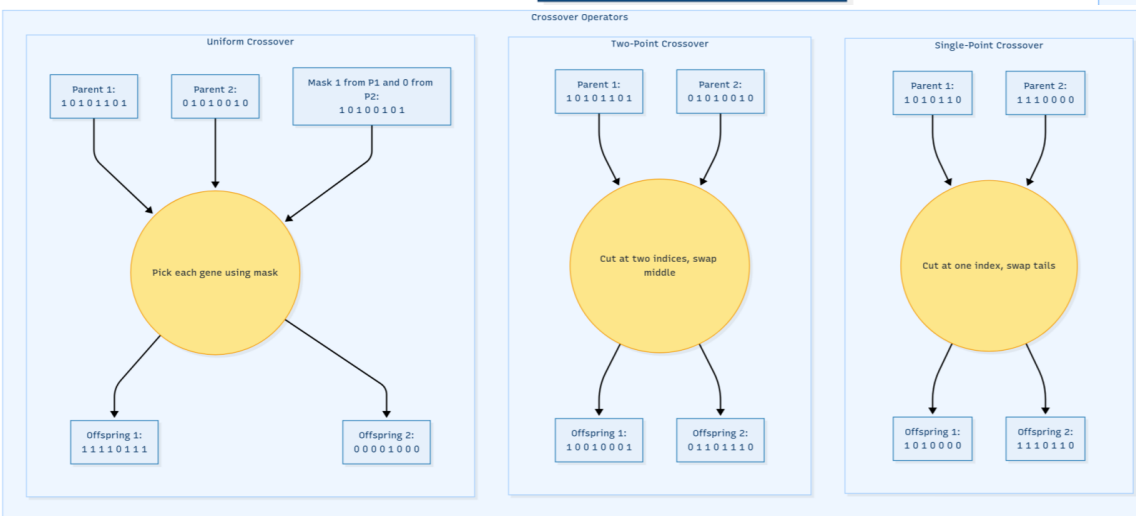
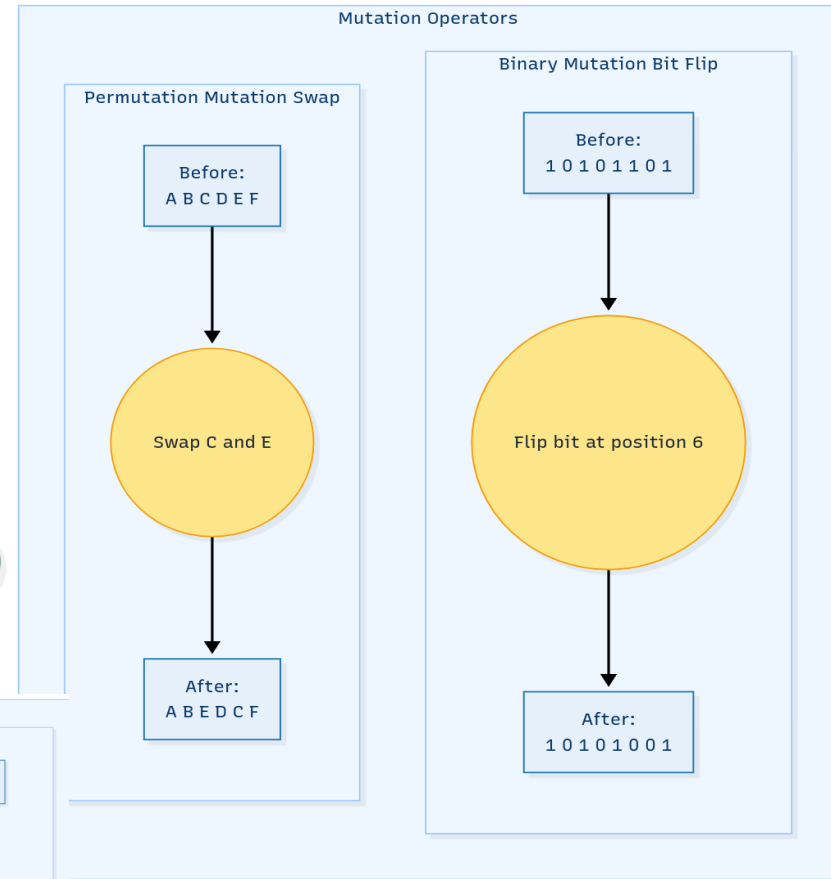
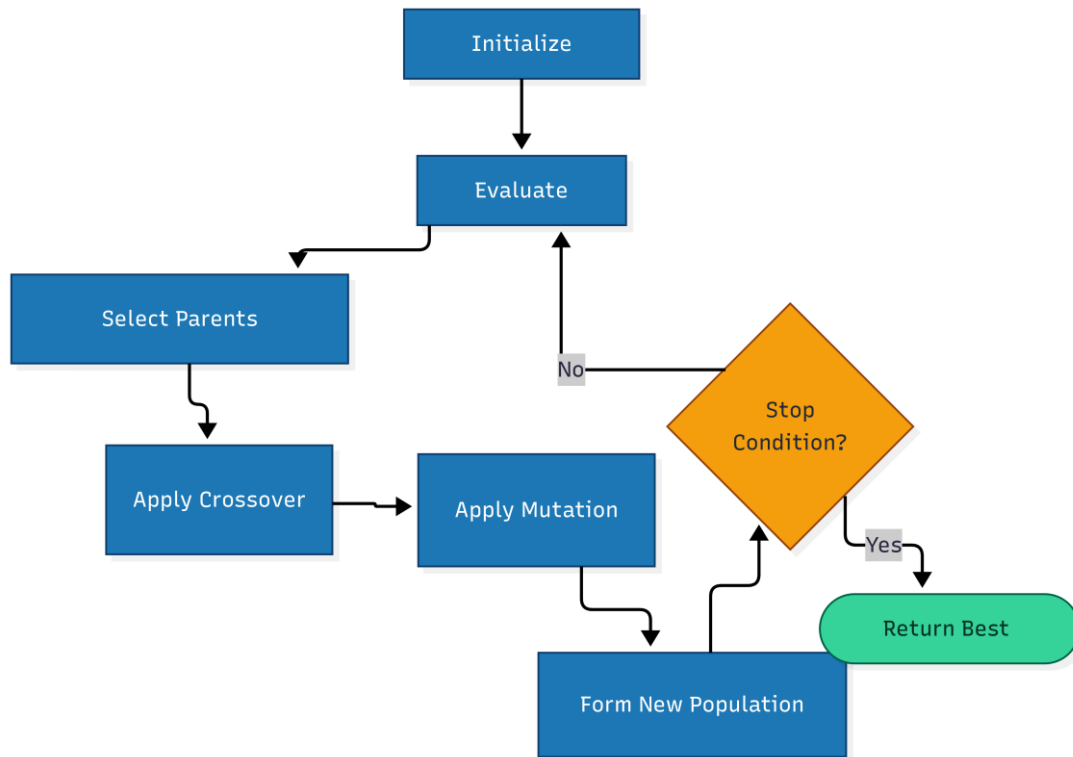


Representation

Chromosomes could be:

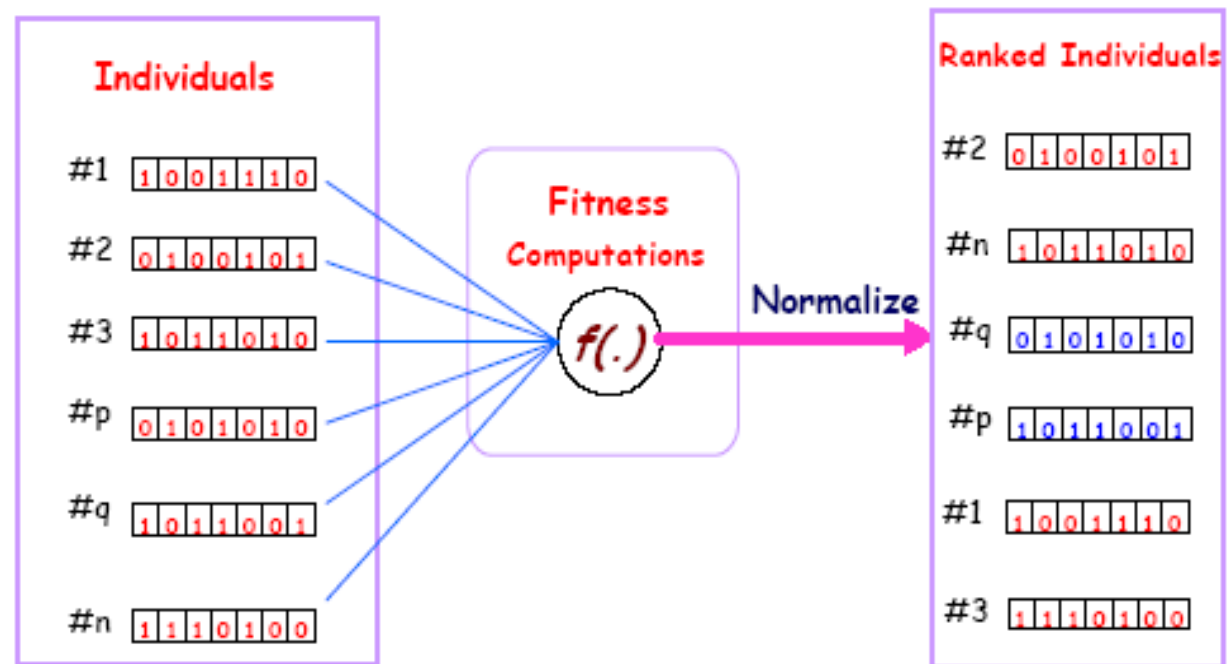
- Bit strings (0101 ... 1100)
- Real numbers (43.2 -33.1 ... 0.0 89.2)
- Permutations of element (E11 E3 E7 ... E1 E15)
- Lists of rules (R1 R2 R3 ... R22 R23)
- Program elements (genetic programming)
- ... any data structure ...

GA Workflow



Fitness Function

- **Fitness = measure of solution quality.**
- Approaches:
 - **Hard penalty** → infeasible = very low fitness.
 - **Soft penalty** → penalize but keep diversity.
 - **Scaling/normalization** to balance selection pressure.
- Example (Knapsack):
- Value = 20, Weight = 9, Capacity = 6.
 - Hard penalty: $20 - 100 \cdot (3) = -280$.
 - Soft penalty: $20 / (1 + 3) = 5$.



Example

- $f(x) = \{ \text{MAX}(x^2) : 0 \leq x \leq 32 \}$
- Encode Solution: Just use 5 bits (1 or 0).
- Generate initial population.

A	0	1	1	0	1
B	1	1	0	0	0
C	0	1	0	0	0
D	1	0	0	1	1

- Evaluate each solution against objective.

Sol.	String	Fitness	% of Total
A	01101	169	14.4
B	11000	576	49.2
C	01000	64	5.5
D	10011	361	30.9

Selection Methods

Roulette wheel:

- Probability $\propto$ fitness.
- Needs non-negative values.

Tournament (k=2 or 3):

- Sample and pick best.
- Stronger pressure with larger k.

Rank-based:

- Use order, not absolute fitness.

Truncation:

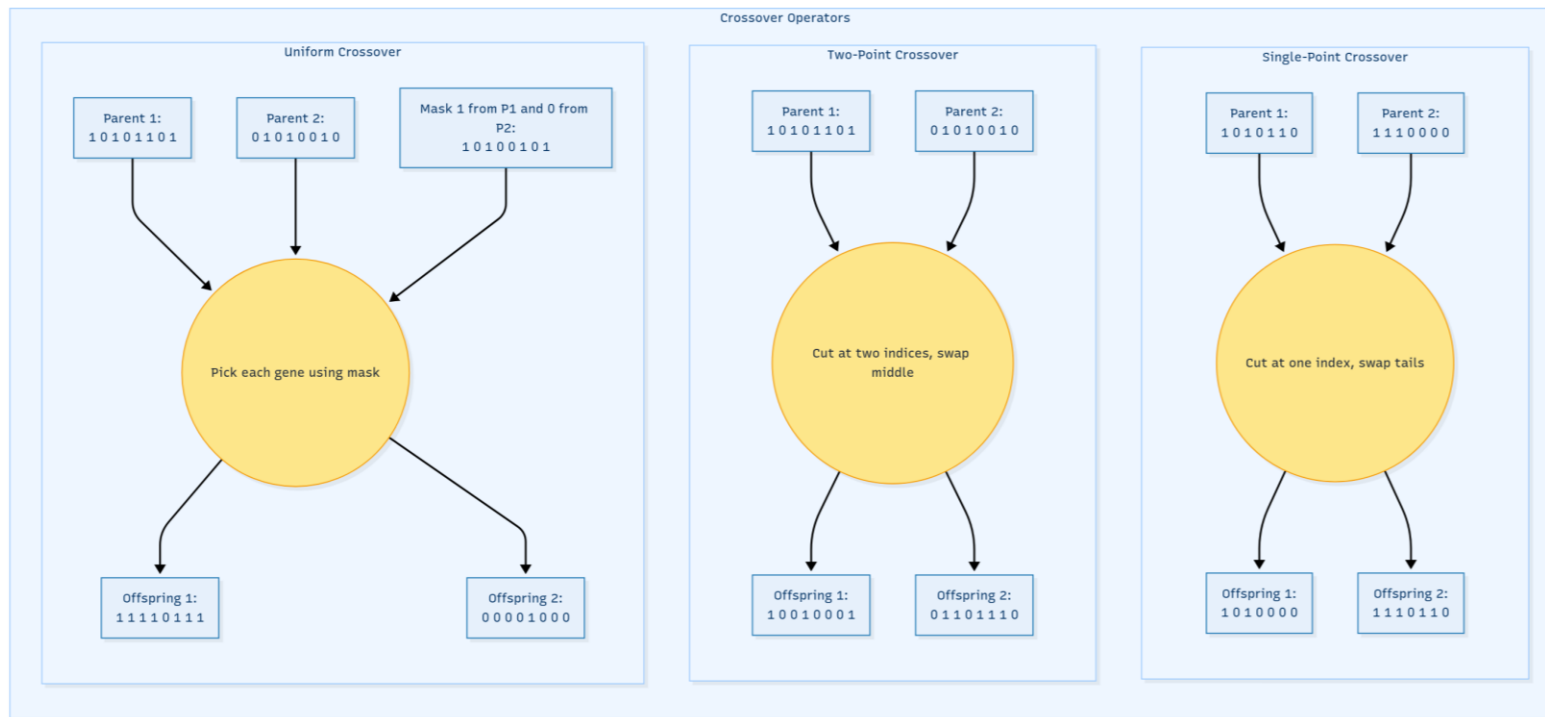
- Keep top % directly.

Elitism:

- Carry over best individuals unchanged.

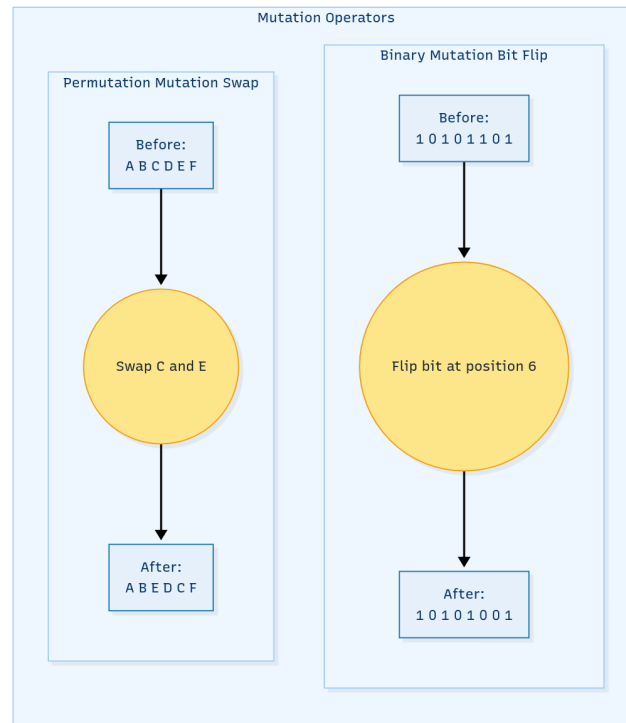
Example Cont'd

- Create next generation of solutions
 - Probability of “being a parent” depends on the fitness.
- Ways for parents to create next generation
 - Reproduction
 - Use a string again unmodified.
 - Crossover
 - Cut and paste portions of one string to another.
 - Mutation
 - Randomly flip a bit.
 - COMBINATION of all of the above.



Crossover

- Purpose: combine parent solutions
- Single-point crossover: cut and swap halves.
- Two-point crossover: swap middle segment.
- Uniform crossover: use random mask per gene.

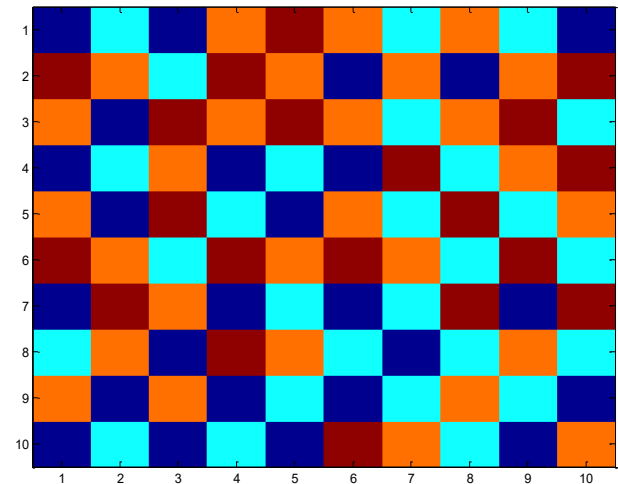
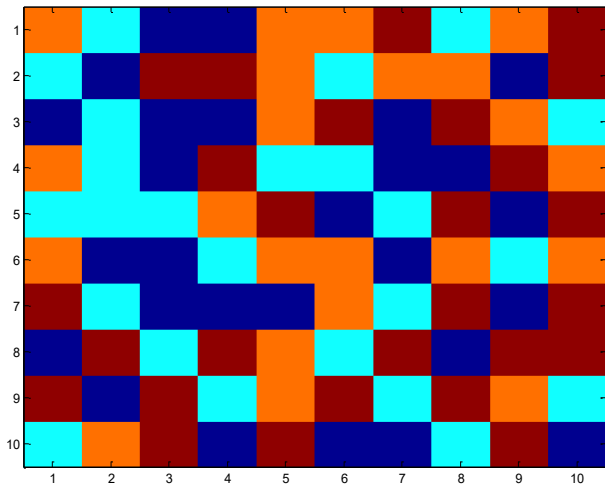


Mutation

- **Bit-flip (binary):** flip gene ($1 \leftrightarrow 0$).
- **Swap (permutation):** swap two positions.
- **Inversion:** reverse a subsequence.
- **Scramble:** shuffle a subsequence.
- Typical **rate**: 1–5%.
- Mutation prevents premature convergence.

Checkboard example

- We are given an n by n checkboard in which every field can have a different colour from a set of four colors.
- Goal is to achieve a checkboard in a way that there are no neighbours with the same color (not diagonal)



Checkboard example Cont'd

- Chromosomes represent the way the checkboard is colored.
- Chromosomes are not represented by bitstrings but by **bitmatrices**
- The bits in the bitmatrix can have one of the four values 0, 1, 2 or 3, depending on the color.
- Crossing-over involves matrix manipulation instead of point wise operating.
- Crossing-over can be combining the parental matrices in a horizontal, vertical, triangular or square way.
- Mutation remains bitwise changing bits in either one of the other numbers.

GA Pseudocode

Choose initial population

Evaluate the fitness of each individual in the population

Repeat

 Select best-ranking individuals to reproduce

 Breed new generation through crossover and mutation (genetic operations) and give birth to offspring

 Evaluate the individual fitnesses of the offspring

 Replace worst ranked part of population with offspring

Until <terminating condition>

Example

SPEA (Strength Pareto EA): multi-objective.

NSGA-II (Non-dominated Sorting GA): maintains Pareto fronts, diversity.

Memetic Algorithms: GA + local search refinement.

Hybrid GAs: combine GA with simulated annealing, greedy search.

Applications of EA/GA

Scheduling:
timetables, jobs

Routing: TSP,
logistics

Engineering:
circuits,
aerodynamics

Machine
learning: feature
selection

Game AI:
strategies

Strengths & Weaknesses

Strengths

- Global search (diversity).
- Flexible, problem-independent.
- Handles non-differentiable problems.

Weaknesses

- Parameter tuning needed (mutation, crossover).
- Can be computationally expensive.
- No optimality guarantee.

The background is a complex fractal pattern. It features deep blue, swirling, fibrous structures that resemble waves or the branching of a tree. These blue structures are interspersed with bright orange, fan-like or petal-like shapes that radiate from various points. The overall effect is a dynamic, organic, and highly detailed visual texture.

OTHER NATURE INSPIRED ALGORITHMS

Other Nature Inspired Algorithms

- **Particle Swarm Optimization (PSO):**

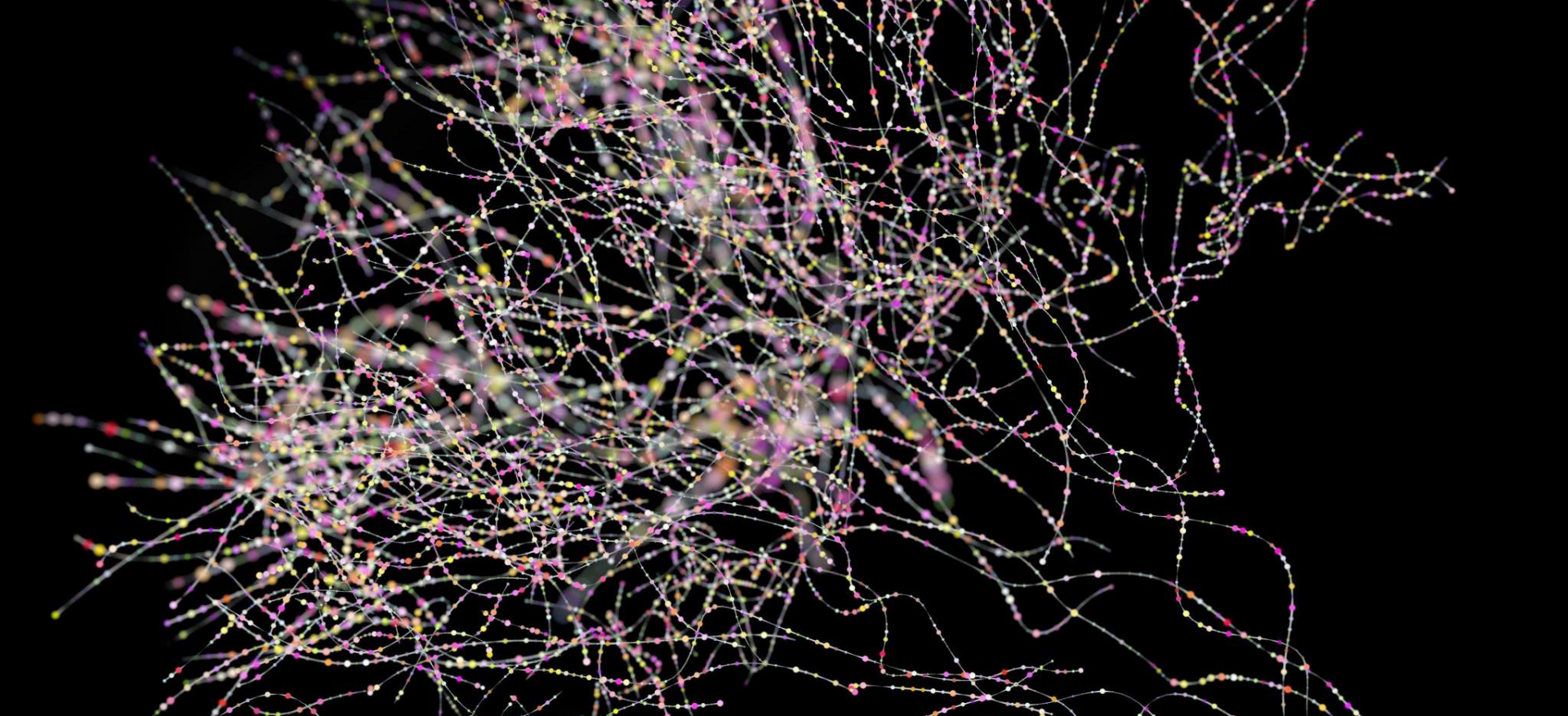
- Inspired by bird flocking.
- Particles move using velocity + memory.
- Good for **continuous** optimization.

- **Ant Colony Optimization (ACO):**

- Pheromone-based path finding.
- Good for **routing/TSP** problems.

- **Evolution Strategies (ES):**

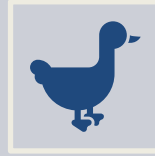
- Real-coded, self-adaptive step sizes.



PARTICLE SWARM OPTIMIZATION

<https://www.youtube.com/watch?v=vhSBqk6SAB4>

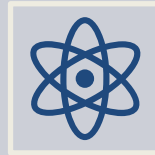
Particle Swarm Optimization (PSO)



Inspired by: flocking birds, schooling fish.



Developed by Kennedy & Eberhart (1995).



Key idea: particles “fly” through solution space, sharing information to converge toward good solutions.



Population-based but *no reproduction/mutation* (contrast with GA).

PSO: Mapping

Inspiration: Bird flocking / fish schooling.

Agents (“particles”) fly in the search space with a **velocity** influenced by:

- Position
- Velocity
- **Personal best** position pbest
- **Global (or neighborhood) best** gbest

Real-world mapping

- **Particle** → candidate solution (vector or permutation)
- **Velocity** → how solution changes (continuous step; or specialized operators for discrete)
- **Fitness** → objective value

How it works

- **Particles = candidate solutions** that move in the search space.
- Each particle remembers:
 - **pbest**: its own best position found so far.
 - **gbest**: the swarm's best position overall.
- Velocity = a balance of:
 - **Inertia** (keep going same way → exploration).
 - **Cognitive pull (c1)**: learn from own past.
 - **Social pull (c2)**: learn from swarm.
- Over time: swarm shifts from wide exploration → focused exploitation.

- **Initialize swarm**

- Random positions (candidate solutions) and small velocities.

- **Repeat until stop**

- Max iterations or convergence; **return gbest.**
 - **Evaluate & set personal bests**
 - $pbest_i \leftarrow$ current position if it improves the particle's own best.
 - **Find global (or neighborhood) best**
 - $gbest \leftarrow$ best $pbest$ across the swarm (or within a topology).
 - **Update velocity**
 - New velocity = (inertia $\times$ old velocity) + (pull to $pbest$) + (pull to $gbest$).
 - **Update position**
 - Move each particle by its velocity; keep inside bounds.

PSO - Key Controls

- **Inertia (w)**
 - High $w \rightarrow$ strong momentum, wide search (exploration).
 - Low $w \rightarrow$ particles slow down, focus locally (exploitation).
- **Cognitive weight ($c1$)**
 - Pull toward **pbest** (self-experience).
 - High $c1 \rightarrow$ particles explore based on their own knowledge.
 - Too high $\rightarrow$ particles “wander alone,” weak collaboration.
- **Social weight ($c2$)**
 - Pull toward **gbest** (social learning).
 - High $c2 \rightarrow$ particles quickly converge to swarm’s best.
 - Too high $\rightarrow$ swarm collapses early to suboptimal solution.

PSO Topologies

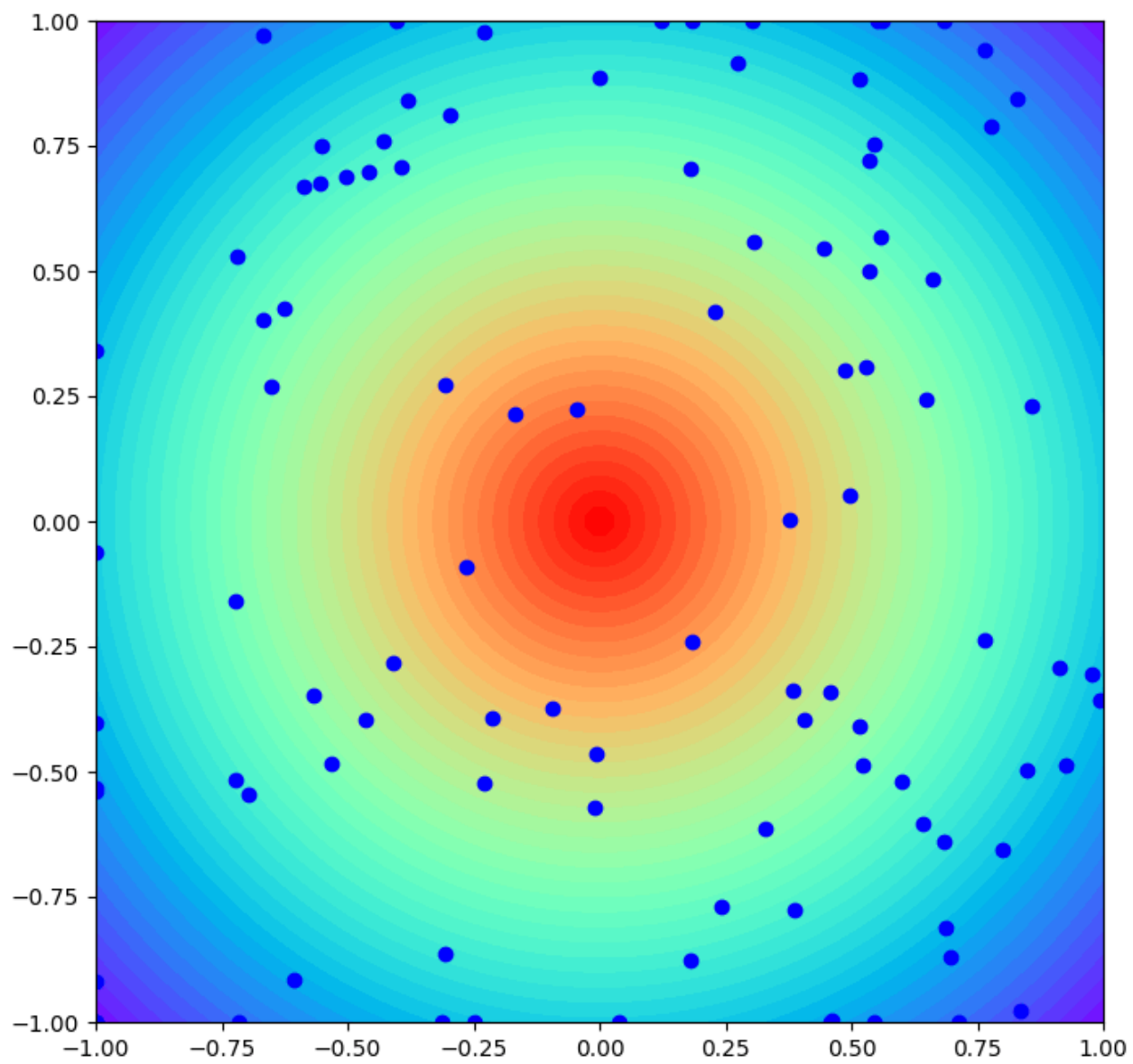
- **Topology = who talks to whom.**
 - Defines the **information-sharing network** among particles.
 - Each particle keeps a **personal best (pbest)** and listens to a **social best**:
 - **gbest topology (star)**: everyone listens to the **global best**.
 - **lbest topology (ring/local)**: each listens to the **best in its neighborhood**.
- Topology controls the **exploration** ↔ **exploitation** balance and **risk of premature convergen**

Global-Best (gbest / Star)

- **How it works**
 - Every particle is attracted to the **single best** pbest found by anyone (the **gbest**).
 - Information spreads **instantly** across the swarm.
- **Behavior**
 - **Very fast convergence** on smooth, unimodal landscapes.
 - **High risk of premature convergence** on rugged/multimodal problems (everyone rushes to the same valley).
- **Pros**
 - Simple to implement; few hyperparameters.
 - Great when the objective is **well-behaved**, low-noise, and you want speed.
- **Cons**
 - Diversity collapses quickly; can get stuck.
 - Sensitive to noisy evaluations (gbest can be a fluke).
- **Mitigations**
 - **Clamp velocities**, schedule **inertia (w)** from high→low, use **random restarts**, or add **noise/jitter**.
 - Try **constriction** or **$c1/c2$** balancing to reduce swarming too hard on early gbest.

Local-Best (lbest / Ring)

- **How it works**
 - Particles arranged in a ring (or small neighborhoods).
 - Each particle is pulled toward the **best pbest in its neighborhood (lbest)**, not the global best.
- **Behavior**
 - **Slower convergence**, but **stronger diversity**.
 - Better at **escaping local minima**; different neighborhoods can explore different basins.
- **Pros**
 - Robust on **multimodal** or **noisy** problems.
 - Naturally maintains **multiple niches** longer.
- **Cons**
 - More iterations to converge; can “circle” if neighborhood is too small or parameters are mild.
- **Design choices**
 - **Neighborhood size (k)**: small k → more diversity; larger k → closer to gbest behavior.
 - **Topology shape**: ring (common), line, grid (von Neumann), small-world.



More on PSO

Pros

- Few parameters, easy to implement.
- Good for continuous optimization.
- Fast convergence in many problems.

Cons

- Risk of premature convergence (too fast).
- Sensitive to parameter tuning.
- Not great for discrete combinatorial problems.

ANT COLONY OPTIMIZATION

[https://www.youtube.com/watch?
v=xGlzOSt9Pjk](https://www.youtube.com/watch?v=xGlzOSt9Pjk)

Ant Colony Optimization (ACO)

Inspired by: ants finding shortest paths using pheromone trails.

Introduced by Dorigo (1992).

Key idea: artificial ants build solutions step by step, guided by pheromone levels + heuristic information

The Foraging Analogy

- Real ants:
 - Explore randomly at first.
 - Deposit pheromones on paths.
 - Shorter paths reinforced more strongly (pheromone concentration grows).
 - Colony converges on shortest path.
- Artificial ants mimic this to solve **combinatorial optimization problems** (TSP, routing, scheduling).



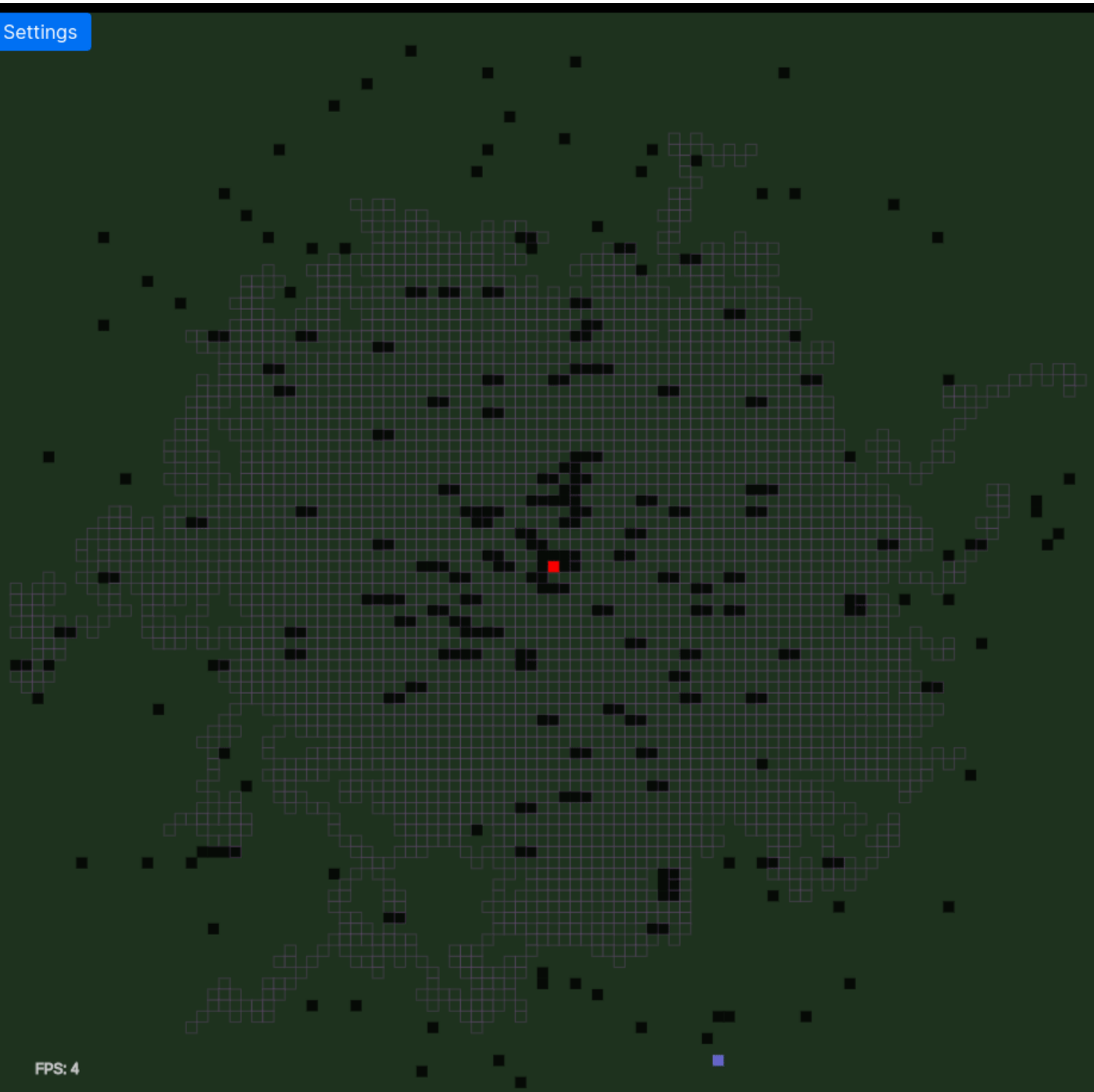
ACO: Intuition & Mapping

- **Inspiration:**
 - Foraging ants lay **pheromones**; shorter/better paths accumulate more pheromone.
- **Search idea:**
 - Many simple agents (ants) construct solutions stochastically, guided by:
 - **Pheromone** trails (what worked well so far)
 - **Heuristic** visibility (instant local desirability)
- **Positive feedback + evaporation** $\Rightarrow$ exploration/exploitation balance.
- **Real-world mapping**
 - **Ants** $\rightarrow$ stochastic constructive agents
 - **Pheromone** τ $\rightarrow$ learned preferences on components (edges, items)
 - **Visibility** η $\rightarrow$ domain heuristic (e.g., $1/\text{distance}$ for TSP; value/weight for knapsack)

Basic Steps in ACO

- **Initialize**
Set small pheromone on all components (e.g., edges/items). Choose α (pheromone weight), β (heuristic weight), ρ (evaporation), #ants, #iterations.
- **For each iteration do:**
 - **Each ant builds a solution**
 - Start from a valid seed; at each step pick the next component **probabilistically** using (pheromone influence) $\times$ (heuristic desirability). Avoid infeasible moves.
 - **Complete all solutions**
 - Stop when a full tour/selection is built (or when no feasible move remains).
 - **Evaluate**
 - Compute solution quality (tour length, total value, etc.).
 - **Evaporate**
 - Reduce pheromone everywhere by a small fraction (prevents stagnation).
 - **Reinforce**
 - Add pheromone along components used by good solutions (e.g., best ant or top-k).
 - **Check stop condition**
 - If not met, **repeat**; otherwise **return the best solution found**.

Settings



FPS: 4

ACO – Key Controls

α (alpha): pheromone importance

- Controls how strongly ants are influenced by pheromone trails.
- **High α** : ants mostly follow strong trails → faster convergence, risk of stagnation.
- **Low α** : ants ignore pheromone → more random exploration.

β (beta): heuristic importance

- Weight given to problem-specific heuristic (e.g., 1/distance in TSP, value/weight in Knapsack).
- **High β** : ants prefer edges/items that look locally attractive.
- **Low β** : ants rely more on pheromone/history than heuristics.

ACO – Key Controls

ρ (rho): evaporation rate

- Fraction of pheromone that evaporates each iteration.
- **High ρ (fast evaporation)**: pheromone fades quickly → prevents early stagnation, promotes exploration.
- **Low ρ (slow evaporation)**: trails last longer → stronger memory, but may lock into suboptimal paths.

#Ants

- More ants = broader exploration but higher runtime cost.
- Typical rule: number of ants $\approx$ number of nodes (TSP) or items (Knapsack).

Elitism

- Extra pheromone deposited by the global best ant each iteration.
- Helps preserve and strengthen the best-so-far solution.
- Risk: too strong elitism → premature convergence.

ACO – Key Controls

Pheromone limits (Min–Max bounds)

- Set minimum and maximum pheromone levels.
- Prevents very weak edges from being permanently ignored (min).
- Prevents domination by one path (max).

Restarts / Diversification

- If convergence is detected (all ants use the same path), restart with fresh pheromone or randomization.
- Ensures long-term exploration.

More on ACO

Pros:

- Well-suited for **discrete combinatorial problems**.
- Exploits both **individual exploration** and **collective memory**.
- Naturally parallelizable.

Cons:

- Computationally expensive (many ants, iterations).
- Risk of stagnation (early bias).
- Parameter tuning required (α , β , ρ).

Summary

Algorithm	Domain	Key Mechanism	Best For
GA	General, discrete/continuous	Reproduction + crossover + mutation	Broad use cases
PSO	Continuous	Velocity + social learning	Real-valued optimization
ACO	Combinatorial	Pheromone-guided probabilistic construction	TSP, routing, scheduling

Summary



EA = family (EP, ES, GA, GP)



GA = most widely used



Key: chromosome, fitness, selection, crossover, mutation



Applications: scheduling, ML, design